

Capitolo 14 - Realizzazione del file system

Status Done

Struttura del file system

Il file system risiede nella memoria secondaria (HDD o NVM). I dischi sono ideali per questo scopo perché permettono di **riscrivere** i dati nella stessa posizione e di accedere direttamente a qualsiasi blocco (accesso casuale).

La stratificazione del file system

Per gestire la complessità, il file system è organizzato a livelli (o strati). Ogni livello fornisce servizi a quello superiore.

1. Controllo dell'I/O (I/O Control):

- È composto dai **driver** e dai gestori di interruzioni.
- Traduce comandi come "leggi il blocco 123" in segnali elettrici e configurazioni di bit per il controller hardware.

2. File system di base (Basic File System):

- Invia comandi generici ai driver per leggere/scrivere blocchi fisici basandosi sulla loro posizione (es. *Cilindro 7, Traccia 2, Settore 10*).
- Gestisce i **buffer** e le **cache** per velocizzare l'accesso ai dati e ai metadati.

3. Modulo di organizzazione dei file (File Organization Module):

- Conosce la struttura logica del file (Blocchi 0, 1, 2...).
- **Traduzione logica-fisica**: traduce il "blocco logico 2" del file nell'indirizzo fisico corrispondente sul disco.
- Gestisce lo **spazio libero** (tiene traccia di quali blocchi sono disponibili per nuovi file).

4. File system logico (Logical File System):

- Gestisce i **metadati** (tutto ciò che non è il contenuto, come permessi e date).
- Gestisce la struttura delle directory.
- Utilizza i **File Control Block (FCB)**, chiamati inode in Unix, che contengono tutte le informazioni vitali sul file (proprietà, posizione del contenuto, ecc.).

Varietà di File System

Perché esistono così tanti tipi di file system? Perché ognuno è progettato per obiettivi diversi (prestazioni vs affidabilità vs portabilità).

• Standard comuni:

- Unix/Linux: UFS, FFS, ext2, ext3 e il moderno ext4.
- Windows: FAT, FAT32 e soprattutto NTFS.
- Supporti ottici: ISO 9660 (per CD-ROM).

• Progetti speciali:

- Google File System (GFS): ottimizzato per gestire enormi quantità di dati su migliaia di dischi.
- FUSE: un sistema che permette di creare e testare nuovi file system nello spazio utente invece che nel kernel, aumentando la flessibilità dello sviluppo.

Vantaggio della stratificazione: riduce la duplicazione del codice (più file system possono usare lo stesso driver di basso livello).

Svantaggio: ogni passaggio tra i livelli può aggiungere un piccolo ritardo (overhead), influenzando le prestazioni.

Operazioni del file system

Panoramica delle strutture dati

Il file system si basa su strutture memorizzate nel disco (persistenti) e strutture in memoria (volatili, caricate all'avvio o al montaggio).

Strutture sul disco

1. **Boot Control Block:** contiene le istruzioni per avviare il SO. È quasi sempre il primo blocco del volume (es. Superblocco in UFS o MFT in NTFS).

2. **Volume Control Block:** memorizza i dettagli della partizione: numero di blocchi, dimensione, quanti blocchi sono liberi e dove si trovano.
3. **Struttura della directory:** organizza i file. In Unix associa nomi di file a numeri di inode.
4. **File Control Block (FCB):** contiene tutti i metadati del file (permessi, dimensione, proprietario). Ogni file ha il suo FCB identificato univocamente.

Strutture in memoria

Servono per velocizzare le operazioni ed evitare accessi continui al disco:

- **Tabella di montaggio:** elenco di tutti i volumi attualmente montati e accessibili.
- **Cache delle directory:** memorizza i percorsi usati di recente per velocizzare la navigazione tra le cartelle.
- **Tabella di sistema dei file aperti:** contiene una copia dell'FCB di ogni file aperto da qualsiasi processo nel sistema.
- **Tabella dei file aperti per processo:** elenco dei file aperti dallo specifico processo, con i relativi permessi e il puntatore alla posizione corrente.

Utilizzo: il ciclo di vita di un file

Apertura (Open)

Quando un processo chiama `open("nome_file")`:

1. **Controllo preventivo:** il kernel cerca nella tabella di sistema se il file è già aperto da qualcun altro. Se sì, crea solo un puntatore nella tabella del processo.
2. **Ricerca su disco:** se il file non è aperto, il sistema cerca il nome nella directory sul disco.
3. **Caricamento:** una volta trovato, l'FCB viene copiato nella tabella di sistema in memoria e viene inizializzato un contatore delle aperture.
4. **Handle/Descrittore:** al processo viene restituito un numero (il file descriptor in Unix o handle in Windows) che userà per tutte le operazioni successive.

Lettura/Scrittura (Read/Write)

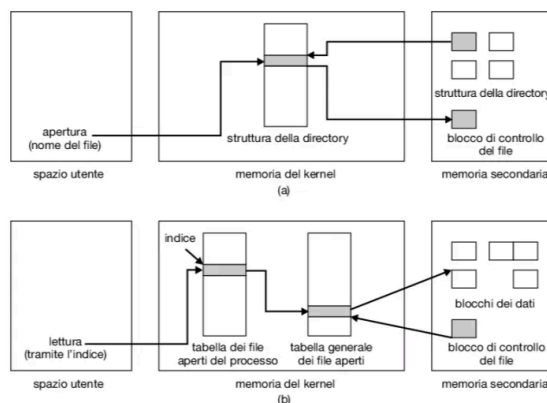
Una volta ottenuto il descrittore, il processo può leggere o scrivere. Il SO usa il puntatore alla posizione corrente salvato nella tabella del processo per sapere da dove iniziare l'operazione, aggiornandolo automaticamente alla fine.

Chiusura (close)

Quando un processo chiama `close()`:

1. Viene rimosso l'elemento della tabella del processo.
2. Il **contatore delle aperture** nella tabella di sistema viene decrementato.
3. **Salvataggio finale:** solo quando il contatore arriva a zero (nessuno usa più il file), i metadati aggiornati vengono riscritti sul disco e il file scompare dalla memoria centrale.

Nota sulle prestazioni: i sistemi moderni (come BSD) usano massicciamente le cache per queste strutture, arrivando a tassi di successo (hit rate) dell'85%, riducendo drasticamente il bisogno di consultare il disco meccanico, che è molto più lento.



Strutture del file system che si mantengono nella memoria; (a) apertura di file; (b) lettura di file.

Realizzazione delle directory

Lista lineare

È il metodo più semplice: i nomi dei file e i puntatori ai dati sono memorizzati uno dopo l'altro in una lista.

- **Funzionamento:**
 - **Creazione:** il sistema deve scorrere l'intera lista per verificare che il nome non sia già presente, poi aggiunge il nuovo elemento in coda.
 - **Cancellazione:** si cerca il file, lo si rimuove e si segna lo spazio come "libero" (usando bit di stato o spostando l'ultimo elemento della lista nella posizione vacante).
- **Vantaggi:** molto facile da programmare.
- **Svantaggi:** *lentezza*. La ricerca è di tipo lineare (bisogna scorrere tutto), il che diventa insostenibile all'aumentare del numero di file.
- **Ottimizzazioni:** l'uso di una lista ordinata (che permette la ricerca binaria) o di alberi bilanciati (come i B-tree) può velocizzare le operazioni, anche se rende più complessa la fase di inserimento.

Tabella hash

Per superare i limiti di velocità della lista lineare, molti file system utilizzano una tabella hash.

- **Funzionamento:**
 1. Il sistema applica una funzione matematica (**hash**) al nome del file (es: "documento.txt").
 2. La funzione restituisce un valore numerico che funge da indice in una tabella.
 3. L'indice punta direttamente alla posizione del file nella lista lineare.
- **Vantaggi:** riduce drasticamente i tempi di ricerca; l'accesso è quasi istantaneo indipendentemente dal numero di file.
- **Svantaggi e sfide:**
 - **Collisioni:** si verificano quando due nomi diversi generano lo stesso valore hash. Si risolvono solitamente con liste concatenate (ogni "cella" dell'hash punta a una piccola lista di file con lo stesso indice).
 - **Dimensione fissa:** spesso queste tabelle hanno una dimensione predefinite. Se la directory cresce troppo, la tabella va ingrandita e tutti gli elementi esistenti devono essere "ricalcolati" (re-hashing), un'operazione onerosa.

Sintesi del Trade-off

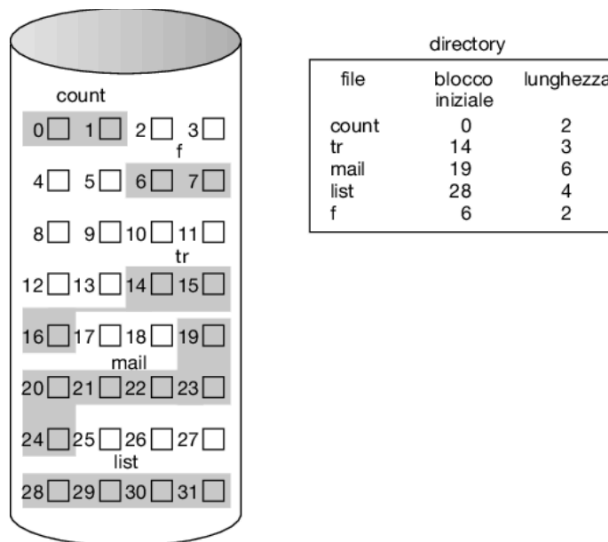
Mentre la **lista lineare** è adatta a directory molto piccole per la sua semplicità, la **tabella hash** (o strutture ad albero più evolute) è indispensabile nei sistemi moderni per garantire prestazioni accettabili quando l'utente gestisce migliaia di file in una singola cartella.

Metodi di allocazione

Allocazione contigua

Ogni file occupa un insieme di blocchi adiacenti sul disco (es. dal blocco 10 al 15).

- **Vantaggi:**
 - **Prestazioni massime:** la testina del disco deve muoversi pochissimo (tempo di ricerca minimo).
 - **Accesso facilitato:** supporta perfettamente sia l'accesso sequenziale che quello diretto (per trovare il blocco i , basta andare a $inizio + i$).
- **Svantaggi:**
 - **Frammentazione esterna:** col tempo, il disco si riempie di piccoli "buchi" liberi troppo piccoli per i nuovi file. Richiede la **deframmentazione** (compattazione), operazione lunga e onerosa.
 - **Difficoltà di crescita:** se un file deve espandersi e i blocchi successivi sono occupati, il file va spostato altrove.
- **Soluzione moderna:** alcuni sistemi usano le estensioni (*extents*): se lo spazio iniziale finisce, si alloca un altro pezzo contiguo altrove e si collegano i due tronconi.

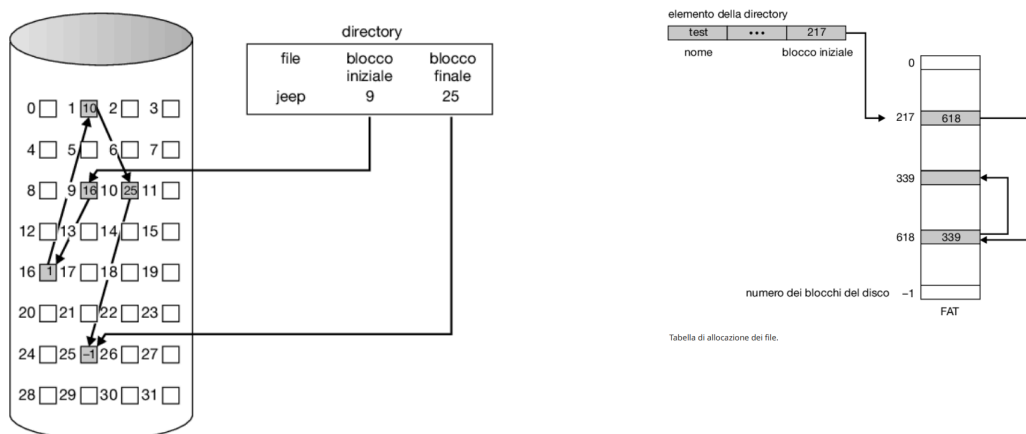


Allocazione contigua dello spazio dei dischi.

Allocazione concatenata

Ogni file è una lista collegata di blocchi sparsi sul disco. Ogni blocco contiene un puntatore al successivo:

- **Vantaggi:**
 - **Nessuna frammentazione esterna:** ogni blocco libero può essere usato.
 - Il file può crescere all'infinito (finché c'è spazio sul disco).
- **Svantaggi:**
 - **Accesso diretto inefficiente:** per leggere il blocco 10, devo prima leggere i 9 precedenti per trovare i puntatori.
 - **Affidabilità:** se un puntatore si corrompe, si perde il resto del file.
 - **Overhead:** parte di ogni blocco è sprecata per memorizzare il puntatore.
- **Variante FAT (File Allocation Table):** usata da MS-DOS. I puntatori non sono nei blocchi dati, ma in una tabella separata all'inizio del disco. Se la FAT è in cache, l'accesso diretto diventa molto più veloce.



Allocazione concatenata dello spazio dei dischi.

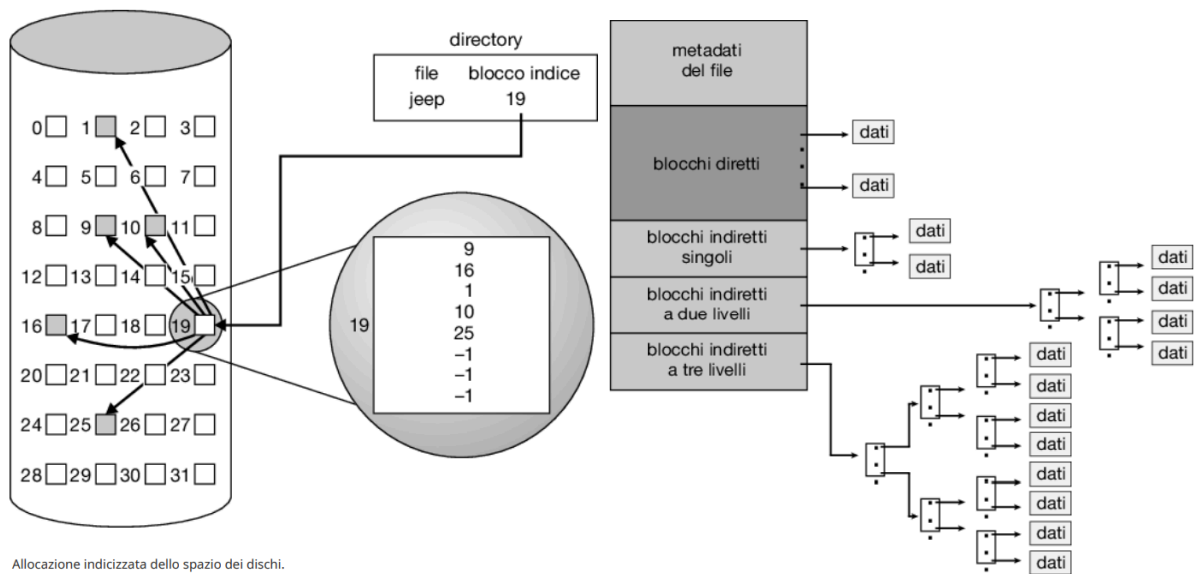
Allocazione indicizzata

Raggruppa tutti i puntatori di un file in un unico luogo chiamato **blocco indice**.

- **Funzionamento:** la directory punta al blocco indice, che è un array di indirizzi ai blocchi dati del file.
- **Vantaggi:** supporta l'accesso diretto in modo efficiente e non soffre di frammentazione esterna.
- **Svantaggi:** spreco di spazio per file piccoli (un file di un solo blocco richiede comunque un intero indice).

- **Meccanismi per file grandi:**

- **Schema concatenato:** un blocco indice punta al successivo.
- **Indice a più livelli:** un indice di primo livello punta a indici di secondo livello.
- **Schema combinato (UNIX Inode):** l'inode contiene 12 puntatori diretti (per file piccoli), un puntatore indiretto singolo, uno doppio e uno triplo. Questo permette di gestire file da pochi KB fino a diversi Terabyte o Exabyte.



Prestazioni e NVM

Le prestazioni dipendono dal tipo di utilizzo:

- **Accesso sequenziale:** l'allocazione concatenata va bene.
- **Accesso diretto:** l'allocazione contigua è la migliore, seguita dall'indicizzata.

Nota sui dispositivi NVM (SSD): poiché non hanno testine meccaniche, il "tempo di ricerca" è nullo. Molti algoritmi classici progettati per ridurre il movimento delle testine sono inutili o inefficiente su NVM. I moderni file system per SSD cercano quindi di ridurre il numero di istruzioni della CPU e semplificare i percorsi dei dati per sfruttare la velocità elettrica del supporto.

Gestione dello spazio libero

Vettore di bit (Bitmap)

È uno dei metodi più comuni: ogni blocco del disco è rappresentato da un singolo bit (1 = libero, 0 = occupato).

- **Vantaggi:** molto semplice ed estremamente efficiente nel trovare il primo blocco libero o sequenze di blocchi contigui grazie a istruzioni hardware veloci.
- **Svantaggi:** richiede molta memoria RAM per dischi grandi. Un disco da 1 TB con blocchi da 4 KB necessita di circa 32 MB di bitmap sempre residente in memoria per essere veloce. Con l'aumento dei Terabyte, questo spazio diventa un problema.

Lista concatenata

Tutti i blocchi liberi sono collegati tra loro. Il SO tiene in memoria un puntatore al blocco libero; questo blocco contiene l'indirizzo del secondo, e così via.

- **Vantaggi:** nessuno spreco di spazio in memoria (si tiene solo un puntatore).
- **Svantaggi: inefficiente** se si deve scorrere la lista (richiede molti I/O su disco). Tuttavia, solitamente serve solo il primo blocco disponibile, quindi l'operazione è rapida.

Raggruppamento (Grouping)

È una variante della lista concatenata: il primo blocco libero memorizza gli indirizzi di **n** altri blocchi liberi. Gli ultimi di questi contengono a loro volta gli indirizzi di un altro gruppo di blocchi liberi.

- **Vantaggio:** permette di trovare rapidamente un gran numero di blocchi liberi con una sola lettura.

Conteggio (Counting)

Sfrutta il fatto che lo spazio viene spesso liberato o allocato in blocchi contigui (estensioni o cluster).

- **Funzionamento:** invece di elencare ogni blocco, si memorizza l'**indirizzo del primo blocco** libero e un **contatore** che indica quanti blocchi contigui lo seguono.
- **Esempio:** (Blocco 10, 5) significa che i blocchi 10, 11, 12, 13 e 14 sono liberi. Riduce drasticamente la lunghezza della lista dello spazio libero.

Mappa di spazio (ZFS Space Maps)

Il file system ZFS di Oracle utilizza una tecnica avanzata per gestire volumi enormi dove le bitmap sarebbero inefficienti.

1. **Metaslab:** divide il disco in centinaia di grandi aree gestibili.
2. **Log delle transazioni:** registra ogni allocazione / liberazione come un log cronologico (tipo "aggiungi blocco X", "rimuovi blocco Y").
3. **Albero bilanciato in RAM:** quando si deve operare su meta-slab, il log viene caricato in un albero bilanciato in memoria per calcolare rapidamente lo stato attuale dello spazio e combinare i blocchi contigui.

TRIM e i dispositivi NVM (SSD)

A differenza degli HDD, le memorie Flash (NVM) **non possono sovrascrivere** i dati direttamente: devono prima cancellare il blocco (operazione lenta che coinvolge molte pagine).

- **Il Problema:** Se il file system cancella un file, l'SSD non lo sa finché non deve scriverci sopra, causando un rallentamento improvviso (detto **write cliff**).
- **La Soluzione (TRIM/Unallocated):** Il file system invia il comando **TRIM** per dire all'SSD: "Questi blocchi non mi servono più".
- **Vantaggio:** Permette al controllore dell'SSD di eseguire la **garbage collection** e la cancellazione dei blocchi in anticipo (nei momenti di inattività), mantenendo le prestazioni di scrittura elevate e uniformi.

Efficienza e prestazioni

Efficienza

L'efficienza riguarda quanto bene viene sfruttato lo spazio e come gli algoritmi riducono il lavoro meccanico del disco.

- **Posizionamento dei metadati:** in UNIX, gli **inode** sono distribuiti in tutto il volume. Il sistema cerca di memorizzare i dati di un file vicino al suo inode per ridurre il movimento della testina (seek time).
- **Dimensione dei cluster:** usare cluster grandi aumenta la velocità ma causa frammentazione interna. Alcuni sistemi (come BSD) usano cluster di dimensioni variabili: grandi per il file voluminosi e piccoli per quelli minuscoli, ottimizzando entrambi gli aspetti.
- **Aggiornamento dei metadati:** mantenere informazioni come "data dell'ultimo accesso" può essere inefficiente, perché ogni semplice lettura di un file costringe il sistema a una scrittura sul disco per aggiornare la data.
- **Evoluzione dei puntatori:**
 - **Puntatori a 32 bit:** limitano i file a 4 GB.
 - **Puntatori a 64 bit:** consentono file enormi, ma occupano più spazio nelle tabelle.
 - **Puntatori a 128 bit (ZFS):** offrono una capacità teoricamente infinita, eliminando la necessità di futuri cambiamenti strutturali.
- **Strutture dinamiche:** i sistemi moderni allocano le tabelle dei file e dei processi dinamicamente, evitando i blocchi del sistema che accadevano in passato quando queste tabelle (a dimensione fissa) si riempivano.

Prestazioni

Le prestazioni si migliorano riducendo il numero di accessi fisici al disco tramite **caching**.

Cache e memoria unificata

- **Cache del controllore:** molti dischi leggono l'intera traccia e la salvano in una memoria locale appena la testina è posizionata.

- **Cache delle pagine (Page Cache):** usa la memoria virtuale per gestire i dati del file come pagine. È più efficiente perché i processi vi accedono direttamente tramite indirizzi virtuali.
- **Buffer cache unificata:** in passato, l'I/O mappato in memoria e le chiamate `read/write` usavano cache diverse, causando il **double caching** (spazio sprecato e dati duplicati). Oggi, i sistemi moderni usano un'unica cache per entrambi i metodi, migliorando coerenza e velocità.

Strategie di ottimizzazione

1. **Scritture sincrone vs asincrone:**
 - **Sincrone:** il processo attende che il dato sia scritto fisicamente sul disco (usate per database e metadati critici).
 - **Asincrone:** il sistema salva i dati nella cache e libera subito il processo. Il disco verrà aggiornato in seguito.
2. **Ottimizzazione per accesso sequenziale:**
 - **Free-behind (rilascio all'indietro):** rimuove una pagina dalla cache non appena si legge la successiva, liberando memoria.
 - **Read-ahead (lettura anticipata):** carica preventivamente le pagine successive a quella richiesta, anticipando i bisogni del processo.
3. **Paradosso della velocità:** grazie alle cache asincrone, scrivere grandi quantità di dati spesso appare più veloce che leggerli, poiché il processo non deve attendere i tempi fisici del disco per considerare l'operazione "conclusa".

Ripristino

Verifica della coerenza

Quando un computer si spegne improvvisamente (crash), alcune modifiche ai file potrebbero essere state scritte sul disco mentre i relativi metadati (directory, inode, liste blocchi liberi) erano ancora nella cache della RAM. Questo crea **incoerenza**.

- **Verifica della coerenza:** è un programma (come `fsck` in Unix o `chkdsk` in Windows) che scansiona l'intero file system per correggere errori (es. un blocco segnato come occupato ma che non appartiene a nessun file).
- **Svantaggi:** è un'operazione estremamente lenta (può richiedere ore per dischi molto grandi) e non garantisce il recupero totale se i danni ai metadati sono gravi.

File system con log delle modifiche (Journaling)

È la tecnica moderna più efficace per garantire un ripristino rapido e sicuro (usata da NTFS, ext4, Veritas).

- **Funzionamento:** ogni modifica ai metadati viene prima scritta sequenzialmente in un file speciale chiamato **log** (o journal). Solo dopo che la scrittura nel log è terminata (**commit**), il sistema aggiorna le strutture dati effettive del disco.
- **Vantaggi nel ripristino:** se il sistema crasha, il kernel controlla solo il log.
 - Se un'operazione è nel log ma non sul disco, la completa.
 - Se un'operazione è incompleta nel log, la annulla.
- **Prestazioni:** contrariamente a quanto si possa pensare, il journaling è molto veloce perché scrivere nel log è un'operazione **sequenziale** (la testina del disco non deve saltare da un punto all'altro).

Altre soluzioni (WAFL e ZFS)

Sistemi come ZFS e WAFL usano una strategia diversa: **non sovrascrivono mai i dati**.

1. **Copy-on-Write:** invece di modificare un blocco esistente, scrivono la nuova versione in un nuovo blocco libero.
2. **Atomicità:** una volta scritti i nuovi dati, il puntatore viene aggiornato per puntare alla nuova versione. Se il sistema crasha durante la scrittura, il vecchio puntatore punta ancora alla versione precedente, perfettamente integra.
3. **Snapshot:** poiché i vecchi blocchi non vengono cancellati subito, è possibile mantenere "istantanee" (snapshot) del file system a un preciso istante nel tempo.

Copie di riserva (Backup)

Nessun sistema di file system può proteggere da un guasto fisico totale del disco (es. rottura meccanica). Sono quindi necessari i backup.

- **Backup completo:** copia di tutti i file presenti sul disco.
- **Backup incrementale:** copia solo i file modificati dall'ultimo backup. È più veloce e richiede meno spazio.

- **Strategia di ripristino:** per recuperare tutto, si ripristina l'ultimo backup completo e, in ordine cronologico, tutti i backup incrementali successivi.

Curiosità: Come citato per la serie *Mr. Robot*, la sicurezza dei dati non dipende solo dai backup, ma anche dalla loro distribuzione geografica (siti diversi) per evitare che un unico evento catastrofico distrugga sia l'originale che le copie.